

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1403
Design

COURSE NAME: Compiler

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
Off Class
Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

*I Pooja sree .M (192521403) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

M.Pooja Sree
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis

Documentation & Presentation	10	Well-structured documentat ion with diagrams, steps, and clarity	Organized documentatio n with required details	Basic documentation with missing details	Poor or incomplete documentation
---------------------------------	----	--	--	---	-------------------------------------

1.) Design a parser to recognize nested if–else statements with compound conditions.

Grammar

$S \rightarrow \text{if}(E) S \text{ else } S$

$S \rightarrow \text{id} = E ;$

$E \rightarrow E \&\& E$

$E \rightarrow E \parallel E$

$E \rightarrow \text{id Relop id}$

$\text{Relop} \rightarrow > | < | ==$

Input

`if((a>b)&&(c<d))`

`x=y;`

`else`

`x=z;`

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the compound conditional statement.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Answer:

LEX CODE:

```
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%

"if"      { return IF; }
"else"    { return ELSE; }

"&&"      { return AND; }
"||"      { return OR; }

">"       { return GT; }
"<"       { return LT; }
```

```
"=="      { return EQ; }
```

```
[a-zA-Z][a-zA-Z0-9]*  { return ID; }
```

```
"="      { return '='; }
```

```
";"      { return ';'; }
```

```
"("      { return '('; }
```

```
")"      { return ')'; }
```

```
[ \t\n]+  ;
```

```
.         { return yytext[0]; }
```

```
%%
```

```
int yywrap()
```

```
{
```

```
    return 1;
```

```
}
```

YACC CODE:

```
%{
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void yyerror(const char *s);
```

```
int yylex();
```

```
%}
```

```
%token IF ELSE ID RELOP AND OR
```

```
%%
```

```
S :
```

```
    IF '(' E ')' S ELSE S
```

```
    | ID '=' ID ';' ;
```

```
E :
```

```
    E AND E
```

```
| E OR E
| '(' E ')'
| ID RELOP ID
;
```

%%

```
void yyerror(const char *s)
{
    printf("\nRejected\n");
}
```

```
int main()
{
    printf("Enter Statement:\n");
    yyparse();
    return 0;
}
```

Sample Input:

```
if((a>b)&&(c<d))
x=y;
else
x=z;
```

COMPILATION:

```
yacc -d parser.y
lex parser.l
gcc lex.yy.c y.tab.c -o parser
./parser
```

OUTPUT:

2.) Design a parser to recognize nested switch-case-default statements.

Grammar

Switch $\rightarrow$ switch(id){CaseList}

CaseList $\rightarrow$ Case CaseList

CaseList $\rightarrow$ Default

Case $\rightarrow$ case num : Stmt break;

Default $\rightarrow$ default : Stmt

Stmt $\rightarrow$ id=num;

Input

switch(choice)

{

case 1:

x=10;

break;

default:

```
x=0;
```

```
}
```

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the switch statement.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

ANSWER:

LEX CODE (switch.l):

```
%{
#include "y.tab.h"
%}

%%

"switch"    { return SWITCH; }
"case"      { return CASE; }
"default"   { return DEFAULT; }
"break"     { return BREAK; }

[a-zA-Z_][a-zA-Z0-9_]* {
    yylval.str = yytext;
    return ID;
}

[0-9]+      {
    yylval.num = atoi(yytext);
    return NUM;
}

"="         { return ASSIGN; }
":"         { return COLON; }
";"         { return SEMICOLON; }
"{"         { return LBRACE; }
"}"         { return RBRACE; }
"("         { return LPAREN; }
")"         { return RPAREN; }
```



```
[ \t\n]+ ; /* Ignore spaces */
```

```
. { return yytext[0]; }
```

```
%%
```

```
int yywrap()
```

```
{  
    return 1;  
}
```

YACC CODE (switch.y):

```
%{
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int yylex();
```

```
void yyerror(char *s);
```

```
%}
```

```
%union
```

```
{  
    char *str;  
    int num;  
}
```

```
%token SWITCH CASE DEFAULT BREAK
```

```
%token ID NUM
```

```
%token ASSIGN COLON SEMICOLON
```

```
%token LBRACE RBRACE LPAREN RPAREN
```

```
%%
```

Program:

Switch

```
{
    printf("\nSwitch Statement Accepted\n");
}
;
```

Switch:

```
SWITCH LPAREN ID RPAREN LBRACE CaseList RBRACE
;
```

CaseList:

```
Case CaseList
|
Default
;
```

Case:

```
CASE NUM COLON Stmt BREAK SEMICOLON
;
```

Default:

```
DEFAULT COLON Stmt
;
```

Stmt:

```
ID ASSIGN NUM SEMICOLON
;
```

%%

void yyerror(char *s)

```
{
    printf("\nSyntax Error: %s\n",s);
    printf("Switch Statement Rejected\n");
}
```

int main()

```

{
    printf("Enter switch statement:\n");
    yyparse();
    return 0;
}

```

SAMPLE INPUT :

```

switch(choice)
{
case 1:
x=10;
break;
default:
x=0;
}

```

OUTPUT:

```

1 %$
2 #include "parser.tab.h"
3 #include <string.h>
4 %$
5
6 "switch" { return SWITCH; }
7 "case" { return CASE; }
8 "default" { return DEFAULT; }
9 "break" { return BREAK; }
10
11 [0-9]+ { yyval.num = atoi(yytext);
12         return NUMBER; }
13 [a-zA-Z_][a-zA-Z0-9_]* {
14     yyval.str = strdup(yytext);
15     return ID; }
16
17 "(" { return LBRACE; }
18 ")" { return RBRACE; }
19 ":" { return COLON; }
20 ";" { return SEMI; }
21 "=" { return ASSIGN; }
22
23 [ \t\r\n]+ { /* ignore whitespace */ }
24
25 %$

```

```

1 %$
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 int yyerror(const char *s);
6
7 %$
8
9 %union {
10     char *str;
11     int num;
12 }
13 %token SWITCH CASE DEFAULT BREAK
14 %token <str> ID
15 %token <num> NUMBER
16 %token LBRACE RBRACE COLON SEMI ASSIGN
17 %$
18
19 Switch : SWITCH '(' ID ')' LBRACE CaseList RBRACE
20         { printf("Accepted\n"); }
21
22 CaseList : Case CaseList
23           | Default
24
25 Case : CASE NUMBER COLON Stmt BREAK SEMI
26
27 Default : DEFAULT COLON Stmt
28
29 Stmt : ID ASSIGN NUMBER SEMI
30
31 %$
32 void yyerror(const char *s)

```

```

PS C:\Users\pooja\Desktop\nested_switch_case> bison -dy parser.y
PS C:\Users\pooja\Desktop\nested_switch_case> flex lexer.l
PS C:\Users\pooja\Desktop\nested_switch_case> gcc parser.tab.c lex.yy.c -o parser.exe -lf
PS C:\Users\pooja\Desktop\nested_switch_case> .\parser.exe
Enter statement:
switch(choice)
{
case 1:
x=10;
break;
default:
x=0;
}
Accepted
PS C:\Users\pooja\Desktop\nested_switch_case>

```

3.) Design a parser to recognize recursive function calls.

Grammar

Call $\rightarrow$ id(ArgList)

ArgList $\rightarrow$ Call

ArgList $\rightarrow$ id

ArgList $\rightarrow \epsilon$

Input

factorial(factorial(n))

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the recursive function call.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

ANSWER:

LEX CODE:

```
%{
#include "parser.tab.h"
#include <string.h>
}%

%%

"("      { return LPAREN; }
")"      { return RPAREN; }

[a-zA-Z_][a-zA-Z0-9_]* {
    yylval.str = strdup(yytext);
    return ID;
}

[ \t\n]+ ; /* Ignore whitespace */

.        { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC CODE:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex();
void yyerror(const char *s);
```

```
%}
```

```
%union
```

```
{  
    char *str;  
}
```

```
%token <str> ID
```

```
%token LPAREN RPAREN
```

```
%%
```

```
Start
```

```
    : Call  
      {  
          printf("Accepted\n");  
      }  
    ;
```

```
Call
```

```
    : ID LPAREN ArgList RPAREN  
    ;
```

```
ArgList
```

```
    : Call  
    | ID  
    |  
    ;
```

```
%%
```

```
void yyerror(const char *s)
```

```
{  
    printf("Rejected\n");  
}
```

```
int main()
```

```
{  
    printf("Enter function call:\n");  
    yyparse();  
    return 0;  
}
```

Compilation Commands:

```
bison -d parser.y
```

```
flex lexer.l
```

```
gcc parser.tab.c lex.yy.c -o parser.exe -lfl
```

INPUT:

factorial(factorial(n))

OUTPUT:

```
1 %{\n2 #include "parser.tab.h"\n3 #include <string.h>\n4 %}\n5\n6 "(" { return LPAREN; }\n7 ")" { return RPAREN; }\n8\n9 [a-zA-Z_][a-zA-Z0-9_]* - {\n10     yyval.str = strdup(yytext);\n11     return ID;\n12 }\n13\n14 [ \\t\\n]+ /* Ignore whitespace */\n15 -\n16 { return yytext[0]; }\n17 %%
```

```
1 %{\n2 #include <stdio.h>\n3 #include <stdlib.h>\n4 #include <string.h>\n5 int yylex(void);\n6 void yyerror(const char *s);\n7 %}\n8\n9 %union{\n10     char *str;\n11 }\n12 %token <str> ID\n13 %token LPAREN\n14 %token RPAREN\n15 %%\n16 Call\n17     : ID LPAREN ArgList RPAREN\n18     { printf("Accepted\\n");\n19     }\n20\n21 ArgList\n22     : Call\n23     | ID\n24     | \n25     ;\n26 %%\n27\n28 void yyerror(const char *s)\n29 {\n30     printf("Rejected\\n");\n31 }\n32\n33 int main()\n34 {\n35     printf("Enter Function Call:\\n");\n36     yyparse();\n37     return 0;\n38 }
```

```
PS C:\\Users\\pooja\\Desktop\\recursive_call> bison -dy parser.y\nPS C:\\Users\\pooja\\Desktop\\recursive_call> flex lexer.l\nPS C:\\Users\\pooja\\Desktop\\recursive_call> gcc parser.tab.c lex.yy.c -o parser.exe -lfl\nPS C:\\Users\\pooja\\Desktop\\recursive_call> .\\parser.exe\nEnter Function Call:\nfactorial(factorial(n))\nAccepted\nPS C:\\Users\\pooja\\Desktop\\recursive_call>
```

4.) Design a parser to recognize nested array indexing expressions.

Grammar

$S \rightarrow id[Index]$

$Index \rightarrow id$

$Index \rightarrow num$

$Index \rightarrow id[Index]$

Input

matrix[row[col]]

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the nested array indexing.
- Display Accepted or Rejected.
- Handle syntax errors.

ANSWER:

LEX CODE:

```
%{
#include "parser.tab.h"
#include <string.h>
#include <stdlib.h>
%}

%%

"["          { return LBRACK; }
"]"          { return RBRACK; }

[0-9]+       {
              yylval.num = atoi(yytext);
              return NUM;
            }

[a-zA-Z_][a-zA-Z0-9_]* {
              yylval.str = strdup(yytext);
              return ID;
            }

[ \t\n]+     ; /* Ignore whitespace */

.            { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC CODE:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);
%}

%union
```

```

{
    char *str;
    int num;
}

%token <str> ID
%token <num> NUM
%token LBRACK RBRACK

%%

S
    : ID LBRACK Index RBRACK
    {
        printf("Accepted\n");
    }
    ;

```

```

Index
    : ID
    | NUM
    | ID LBRACK Index RBRACK
    ;

```

%%

```

void yyerror(const char *s)
{
    printf("Rejected\n");
}

```

```

int main()
{
    printf("Enter expression:\n");
    yyparse();
    return 0;
}

```

COMPILATION:

```

bison -dy parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser.exe -lfl

```

INPUT:

```

matrix[row[col]]

```


OUTPUT:

```
1 %{
2 #include "parser.tab.h"
3 #include <string.h>
4 %}
5
6 "[" { return LBRACK; }
7 "]" { return RBRACK; }
8
9 "[0-9]+" { yylval.num = atoi(yytext);
10          return NUM; }
11
12 "[a-zA-Z_][a-zA-Z0-9_]*" {
13     yylval.str = strdup(yytext);
14     return ID; }
15
16 "[\t\n\r]+" { /* ignore whitespace */ }
17
18 %}
19
20 %token <str> ID
21 %token <num> NUM
22 %token LBRACK RBRACK
23
24 %union{
25     char *str;
26     int num;
27 }
28
29 %start S
30 %type S ID Index
31 %type S ID NUM
32 %type S ID LBRACK Index RBRACK
33
34 %}
35
36 void yyerror(const char *s)
37 {
38     printf("Rejected\n");
39 }
40
41 int main()
42 {
43     printf("Enter expression:\n");
44     yyparse();
45     return 0;
46 }
```

```
PS C:\Users\pooja\Desktop\nested_array_index> bison -dy parser.y
PS C:\Users\pooja\Desktop\nested_array_index> flex lexer.l
PS C:\Users\pooja\Desktop\nested_array_index> gcc parser.tab.c lex.yy.c -o parser.exe -lfl
PS C:\Users\pooja\Desktop\nested_array_index> .\parser.exe
Enter expression:
matrix[row[col]]
Accepted
PS C:\Users\pooja\Desktop\nested_array_index>
```

5.) Design a parser to recognize pointer arithmetic expressions.

Grammar

$E \rightarrow id + num$

$E \rightarrow id - num$

$E \rightarrow *id$

$E \rightarrow \&id$

Input

ptr+5

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.
- Validate the pointer arithmetic expression.
- Display Accepted or Rejected.
- Handle syntax errors.

ANSWER:

LEX CODE:

```
%{
```

```

#include "parser.tab.h"
#include <string.h>
#include <stdlib.h>
%}

%%

"+"          { return PLUS; }
"_"          { return MINUS; }
"*"          { return MUL; }
"&"          { return ADDR; }

[0-9]+       {
                yylval.num = atoi(yytext);
                return NUM;
            }

[a-zA-Z_][a-zA-Z0-9_]* {
                yylval.str = strdup(yytext);
                return ID;
            }

[ \t\n]+     ; /* Ignore whitespace */

.            { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}

```

YACC CODE:

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);
%}

%union
{
    char *str;
    int num;
}

```

```
%token <str> ID
%token <num> NUM
%token PLUS MINUS MUL ADDR
```

```
%%
```

```
E
: ID PLUS NUM
{
    printf("Accepted\n");
}
| ID MINUS NUM
{
    printf("Accepted\n");
}
| MUL ID
{
    printf("Accepted\n");
}
| ADDR ID
{
    printf("Accepted\n");
}
;
```

```
%%
```

```
void yyerror(const char *s)
{
    printf("Rejected\n");
}
```

```
int main()
{
    printf("Enter expression:\n");
    yyparse();
    return 0;
}
```

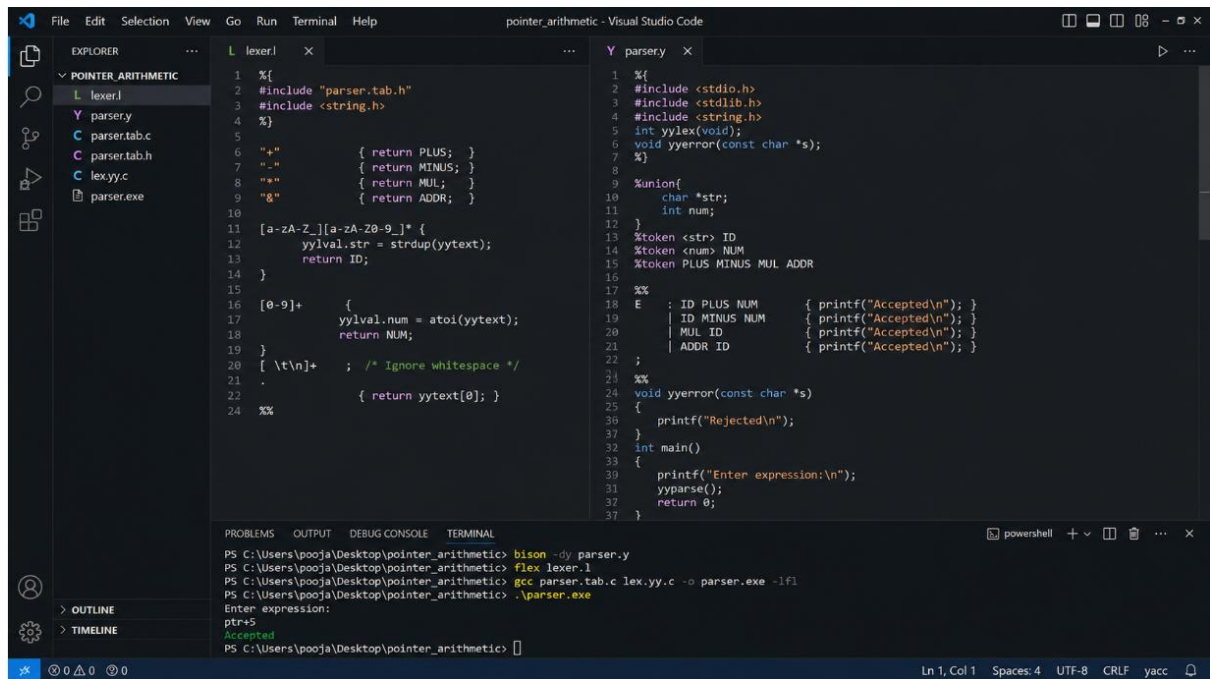
COMPILATION:

```
bison -dy parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser.exe -lfl
```

INPUT:

ptr+5

OUTPUT:



```
1  %{
2  #include "parser.tab.h"
3  #include <string.h>
4  %}
5
6  "+"      { return PLUS; }
7  "-"      { return MINUS; }
8  "*"      { return MUL; }
9  "&"      { return ADDR; }
10
11  [a-zA-Z_][a-zA-Z0-9_]* {
12      yylval.str = strdup(yytext);
13      return ID;
14  }
15
16  [0-9]+    {
17      yylval.num = atoi(yytext);
18      return NUM;
19  }
20  [ \t\n]+ ; /* Ignore whitespace */
21  .        { return yytext[0]; }
22  %%
```

```
1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  int yylex(void);
6  void yyerror(const char *s);
7  %}
8
9  %union{
10     char *str;
11     int num;
12 }
13 %token <str> ID
14 %token <num> NUM
15 %token PLUS MINUS MUL ADDR
16
17 %%
18 E : ID PLUS NUM      { printf("Accepted\n"); }
19   | ID MINUS NUM     { printf("Accepted\n"); }
20   | MUL ID            { printf("Accepted\n"); }
21   | ADDR ID          { printf("Accepted\n"); }
22   ;
23
24 void yyerror(const char *s)
25 {
26     printf("Rejected\n");
27 }
28
29 int main()
30 {
31     printf("Enter expression:\n");
32     yyparse();
33     return 0;
34 }
```

```
PS C:\Users\pooja\Desktop\pointer_arithmetic> bison -dy parser.y
PS C:\Users\pooja\Desktop\pointer_arithmetic> flex lexer.l
PS C:\Users\pooja\Desktop\pointer_arithmetic> gcc parser.tab.c lex.yy.c -o parser.exe -lfl
PS C:\Users\pooja\Desktop\pointer_arithmetic> .\parser.exe
Enter expression:
ptr+5
Accepted
PS C:\Users\pooja\Desktop\pointer_arithmetic>
```